DESIGN AND TRAIN AI MODELS FOR PERFORMANCE PREDICTION

Leashan Marino Sylvester

IT22311368

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

DECLARATION OF CANDIDATE AND SUPERVISOR

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature: _____

Date: _____

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor: _____

Date: _____

DEDICATION

I dedicate this dissertation to my beloved family, whose patience, encouragement, and quiet sacrifice have been the foundation of every chapter that follows. To my parents, who taught me the value of disciplined work and curiosity, and to my siblings and closest friends, who held me up through the long debugging nights and the more demanding implementation phases of this project — this work belongs to you as much as it does to me. Their unwavering belief in my abilities sustained me through the difficult moments of model retraining, edge-case investigation, and threshold calibration that this dissertation describes.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mr. Pradeep K.W. Abeygunawardhana, and my co-supervisor, Ms. Akshi De Silva, for their invaluable guidance, technical insight, and continuous support throughout this research project. Their expertise in applied machine learning, statistical inference, and human-computer interaction shaped the design of the prediction model, the fatigue scoring formulation, and the threshold-based alert subsystem that together form the subject of this dissertation. Their feedback during the calibration of the trend-classification slopes and the fatigue-component weights was particularly important, and their consistent availability for design reviews helped resolve several non-obvious questions about the appropriate confidence-interval construction for short time series.

I am deeply grateful to the Sri Lanka Institute of Information Technology for providing the academic environment, laboratory facilities, and computing resources that enabled this research. The Faculty of Computing has been a supportive community throughout my undergraduate studies, and the department staff have always been available to assist with administrative and technical matters. The availability of stable workstations and reliable network access was essential for the model evaluation work and the latency benchmarking reported in Chapter 4.

I would also like to thank my fellow NeuroMotion team members: Mathumitha Gnanasekaran (IT21379338), Leashan Marino Sylvester (IT22311368), Manulakshan Yogarajah (IT22283962), and Giritharan Jude Jawakker (IT22330864). Each member contributed unique expertise that enriched the overall system. The integration points between the prediction-and-alert component described here and the upstream pose estimation, biomechanical analysis, and visualisation modules required close collaboration, careful contract negotiation around the per-frame status object, and patient cross-team debugging during the integration phases. The team spirit and mutual support throughout the project timeline made the experience both productive and enjoyable.

Special thanks to the volunteer participants who agreed to use the system during the user-acceptance testing phase. Their session data formed the basis for the model validation, the calibration of the fatigue-component weights, and the empirical tuning of the four-second voice-alert rate limit. Their candid feedback on alert frequency, fatigue indication clarity, and prediction interpretability shaped many of the decisions documented in Chapter 3.

Finally, I extend my heartfelt appreciation to my family and friends for their unwavering support and patience during the more demanding phases of this research. Their belief in my abilities and their understanding during the long working hours have made this achievement possible.

ABSTRACT

Athletes who train without access to laboratory-grade biomechanics equipment receive feedback that is, by current consumer-software standards, retrospective and unconditioned: a static post-session report, a numeric counter, and at best a coarse trend line that says nothing about the uncertainty around it. The intelligence layer required to convert that retrospective surface into a forward-looking, calibrated, actionable feedback channel one that anticipates the next session before it begins, detects accumulating fatigue before the athlete does, and intervenes within four seconds of a biomechanical fault has not been satisfactorily delivered by any open-source platform that operates on consumer hardware. This component of the NeuroMotion system addresses that gap by combining three tightly integrated subsystems within a single inference and alerting layer. The feature engineering pipeline extracts three primary predictors from each historical session record exercise speed in repetitions per minute, training volume normalised against a fifty-repetition baseline, and form quality computed as one minus the bad-move rate — from which two composite scores are derived, an endurance score weighted as zero point five five times normalised volume plus zero point four five times normalised speed, and a performance rating weighted as zero point four five times normalised speed, zero point three zero times normalised volume, and zero point two five times form quality, all clamped to valid ranges by a universal bound-enforcement function that prevents numerical overflow from sparse or noisy session histories. The core predictive model is a scikit-learn LinearRegression estimator with a fully self-contained NumPy least-squares fallback, fitted independently across three parallel time series with the session index as the temporal input variable, such that the longitudinal trajectory of speed, endurance, and rating is learned separately and the next-session forecast, the trend slope coefficient, and a 95% confidence interval constructed as one point nine six standard deviations of in-sample residuals are all surfaced to downstream consumers. Trend classification applies fixed slope thresholds improving above zero point one five repetitions per minute per session for speed and zero point five repetitions per session for count, declining below their negatives, stable otherwise and three standard regression error metrics, root mean squared error, mean absolute error, and the coefficient of determination, are computed for each parallel target and persisted in a PredictionResult dataclass alongside three training-load-scenario forecasts at eighty, one hundred, and one hundred twenty percent that incorporate a fatigue

discount factor of one minus zero point two five times the load excess. The continuous-prediction pipeline is event-driven: every detected repetition triggers an update_performance_prediction call that re-queries the database, supplies the latest live-session metrics to compute_performance_prediction, and persists the returned PredictionResult to Streamlit session state, ensuring the displayed forecast tracks the live session without requiring it to end. The fatigue-detection module aggregates four post-prediction signals into a scalar fatigue score on a zero-to-one-hundred scale: a trend-direction factor of up to thirty times the trend strength when declining, a prediction-variance factor of up to twenty-five points scaling with the complement of average R squared, a prediction-error factor of up to twenty points scaling with the sum of speed and endurance RMSE, and a performance-drop factor of thirty times the trend strength applied when the trend is declining and intentionally double-weighting the strongest fatigue indicator. The resulting score is discretised into four levels — Minimal, Low, Moderate, High — each mapped to a colour and an intervention recommendation. The threshold-based alert subsystem operates at two temporal scales: at the frame level, fixed biomechanical thresholds of fifteen degrees for knee valgus, thirty degrees for anterior trunk lean, and zero point one normalised units for knee-over-toe displacement gate a binary danger flag that invokes a trigger_voice_alert function, which injects a JavaScript SpeechSynthesisUtterance via Streamlit components.html with a four-second inter-alert rate-limit guard; at the session level, the fatigue-score thresholds govern a tiered intervention display with colour-coded severity indicators. Evaluation across forty-five test sessions and twelve athletes shows R squared of zero point eight zero for performance rating, ninety-three point one percent confidence-interval coverage against held-out next-session observations, eighty-four point six percent agreement between computed and self-reported fatigue tier, sub-ten-millisecond frame-level alert latency, and ninety-five point eight percent voice-alert delivery within the rate-limit envelope. The component therefore demonstrates that a coordinated, calibrated, real-time machine-learning and threshold-based alerting layer for athletic training can be built entirely on open-source tooling and consumer hardware.

Keywords: performance prediction, linear regression, confidence interval, fatigue detection, threshold-based alerting, Web Speech API, biomechanical thresholds, event-driven inference, scikit-learn, NumPy fallback

TABLE OF CONTENTS

Declaration of Candidate and Supervisor	i
Dedication	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
List of Appendices	ix
CHAPTER 1: INTRODUCTION	1
1.1 Background and Literature Survey	1
1.2 Research Gap	6
1.3 Research Problem	9
1.4 Research Objectives	10
CHAPTER 2: LITERATURE REVIEW	12
2.1 Machine Learning for Athletic Performance Prediction	12
2.2 Linear Versus Nonlinear Models for Short Time Series	14
2.3 Confidence Interval Construction Methods	16
2.4 Fatigue Detection in Sports Science	18
2.5 Biomechanical Fault Thresholds in the Clinical Literature	20
2.6 Voice Alert Systems and Auditory Rate Limiting	22
2.7 Event-Driven Inference Pipelines	23
2.8 Summary	24
CHAPTER 3: METHODOLOGY	26
3.1 System Architecture and Data Flow	26
3.2 Feature Engineering Pipeline	28
3.3 Universal Bound-Enforcement Function	30

3.4 Composite Performance Scores 31
3.5 Linear Regression Model Specification 33
3.6 NumPy Least-Squares Fallback 35
3.7 Three Parallel Time Series Modelling 36
3.8 Trend Slope Coefficient Extraction 38
3.9 Ninety-Five Percent Confidence Interval Construction 39
3.10 Trend Classification Thresholds 41
3.11 Regression Error Metrics 43
3.12 PredictionResult Dataclass 44
3.13 Training Load Scenarios and Fatigue Discount Factor 46
3.14 Event-Driven Update Pipeline 48
3.15 Streamlit Session State Persistence 50
3.16 Fatigue Detection Mathematics Overview 51
3.17 Trend-Direction Factor 53
3.18 Prediction-Variance Factor 54
3.19 Prediction-Error Factor 55
3.20 Performance-Drop Factor with Double Weighting 56
3.21 Fatigue Level Discretisation and Colour Coding 58
3.22 Frame-Level Biomechanical Threshold Subsystem 60
3.23 trigger_voice_alert Function and JavaScript Bridge 62
3.24 Four-Second Inter-Alert Rate Limit 64
3.25 Tiered Intervention Recommendations 65
3.26 Commercialisation Considerations 66
3.27 Testing and Validation Strategy 68
CHAPTER 4: RESULTS AND DISCUSSION 70
4.1 System Performance Overview 70
4.2 Per-Target Regression Error Metrics 72
4.3 Trend Classification Accuracy 74
4.4 Confidence Interval Empirical Coverage 75

4.5 Training Load Scenario Validation 76
4.6 Fatigue Score Correlation with Subjective Rating 78
4.7 Fatigue Level Classification Agreement 79
4.8 Frame-Level Alert Latency 80
4.9 Voice Alert Delivery Within Rate Limit 81
4.10 Event-Driven Pipeline Overhead 82
4.11 Comparative Analysis 83
4.12 Discussion 85
4.13 Summary of My Individual Contribution 87
CHAPTER 5: CONCLUSION 89
5.1 Summary 89
5.2 Achievements 90
5.3 Limitations 91
5.4 Future Work 92
Reference List 94
Appendices 97

LIST OF FIGURES

Figure 1.1: Conceptual placement of the prediction and alerting layer within NeuroMotion 5
Figure 3.1: Component architecture and data flow 27
Figure 3.2: Feature engineering pipeline from raw session record to engineered features 29
Figure 3.3: Universal bound-enforcement function applied across the pipeline 30
Figure 3.4: Three parallel linear regressions on speed, endurance, and rating 37
Figure 3.5: Trend slope and 95% confidence interval visualisation 40
Figure 3.6: Trend classification slope thresholds 42
Figure 3.7: Training load scenarios at 80%, 100%, and 120% with fatigue discount 47
Figure 3.8: Event-driven update pipeline triggered after every detected repetition 49
Figure 3.9: Fatigue score component contributions and aggregation 52
Figure 3.10: Performance-drop factor double-weighting illustration 57
Figure 3.11: Fatigue level discretisation with colour-coded severity tiers 59
Figure 3.12: Frame-level biomechanical threshold gating logic 61
Figure 3.13: trigger_voice_alert call sequence with rate-limit guard 63
Figure 4.1: Predicted versus actual next-session performance rating scatter 73
Figure 4.2: Confidence interval coverage across forecast targets 75
Figure 4.3: Fatigue score versus subjective fatigue rating Bland-Altman plot 78
Figure 4.4: Frame-level alert latency distribution 81

LIST OF TABLES

Table 1.1: Comparison of athletic prediction and alerting platforms 4
Table 2.1: Comparison of model families for short athletic time series 15
Table 2.2: Confidence interval methods for short series 17
Table 2.3: Biomechanical threshold values from the clinical literature 21
Table 3.1: Engineered feature definitions 29
Table 3.2: Composite performance score weights 32
Table 3.3: Trend classification slope thresholds 42
Table 3.4: Regression error metric definitions 43
Table 3.5: PredictionResult dataclass field reference 45
Table 3.6: Training load scenarios and fatigue discount factors 47
Table 3.7: Fatigue score component weights and contribution caps 52
Table 3.8: Fatigue level severity bands and colour mapping 59
Table 3.9: Frame-level biomechanical thresholds 61
Table 3.10: Tiered intervention recommendation map 65
Table 3.11: Commercialisation technology stack 67
Table 4.1: System-level performance metrics 71
Table 4.2: Per-target regression error metrics 73
Table 4.3: Frame-level alert latency statistics 81
Table 4.4: Comparative analysis against representative existing tools 84

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
BPM	Beats Per Minute
CI	Confidence Interval
CPU	Central Processing Unit
EMA	Exponential Moving Average
JS	JavaScript
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Squared Error
OLS	Ordinary Least Squares
RMSE	Root Mean Squared Error
RPM	Repetitions Per Minute
R^2	Coefficient of Determination
SD	Standard Deviation
SE	Standard Error
SQL	Structured Query Language
TTS	Text-to-Speech
UI	User Interface

LIST OF APPENDICES

Appendix A: Feature Engineering Formulas 97

Appendix B: Fatigue Scoring Formulas and Component Pseudocode 99

Appendix C: Linear Regression and Confidence Interval Pseudocode 101

Appendix D: trigger_voice_alert Pseudocode 103

Appendix E: Biomechanical Threshold Derivation Notes 104

CHAPTER 1: INTRODUCTION

1.1 Background and Literature Survey

The training environments of school, university, and recreational athletes have changed dramatically over the past decade, but the intelligence that surrounds the athlete during a session has not kept pace with the underlying detection technology. Modern pose estimation models can extract joint coordinates from a webcam at sub-100 millisecond latency on commodity hardware, and modern web browsers can synthesise context-aware spoken text in fewer than two hundred milliseconds, yet the layer that sits between the raw detection output and the athlete — the layer that asks not only what is happening right now but what will happen next, what has happened over the past several sessions, and what action the athlete should take in response — is largely absent from the open-source ecosystem and only partially present in subscription-grade platforms.

This dissertation describes the design, implementation, and evaluation of that intelligence layer for the NeuroMotion system. The component combines three tightly integrated subsystems within a single inference and alerting pipeline. The first subsystem is a feature engineering and predictive modelling stack that extracts three primary predictors — exercise speed, normalised training volume, and form quality — from each historical session record, derives two composite performance scores from those predictors, and fits three parallel linear regression models that produce next-session forecasts together with calibrated confidence intervals and trend classifications. The second subsystem is a fatigue detection module that aggregates four post-prediction signals into a scalar fatigue score on a zero-to-one-hundred scale, discretises the score into four severity tiers, and maps each tier to a colour-coded intervention recommendation. The third subsystem is a threshold-based alert mechanism that operates at two temporal scales: at the frame level, fixed biomechanical thresholds for knee valgus, anterior trunk lean, and knee-over-toe displacement gate a binary danger flag that invokes a voice alert through the browser's Web Speech API with a four-second rate-limit guard; at the session level, the fatigue-score thresholds govern the tiered intervention display.

The literature on machine learning for athletic performance prediction is extensive but bifurcated. One branch has focused on laboratory-grade prediction with rich physiological and

biomechanical inputs, where complex models such as gradient-boosted trees, random forests, and neural networks routinely outperform simpler baselines. The other branch has focused on consumer-grade prediction from sparse session data, where the literature consistently reports that simple linear models with explicit uncertainty bounds outperform more complex alternatives that overfit the limited training history. The present work sits firmly in the second branch and inherits the methodological consensus that supports it: scikit-learn LinearRegression as the primary estimator, with a NumPy least-squares fallback for environments in which scikit-learn is unavailable, and ninety-five-percent confidence intervals constructed as one point nine six standard deviations of the in-sample residuals.

The literature on fatigue detection in software-based athletic systems has converged on three families of approach: physiological signal analysis, behavioural pattern analysis, and self-report. Physiological approaches require dedicated wearable hardware and are therefore inconsistent with the consumer-hardware accessibility constraint of the present project. Self-report is unreliable in motivated training contexts because athletes systematically under-report fatigue. Behavioural pattern analysis — in particular, analysis of the trend, variance, and recent delta of model-derived performance metrics — is therefore the most appropriate approach, and it is the approach adopted in this work.

The literature on threshold-based biomechanical fault detection draws from sports medicine and rehabilitation engineering. The fifteen-degree threshold for knee valgus is consistent with the published recommendations on landing mechanics and anterior cruciate ligament injury prevention. The thirty-degree threshold for anterior trunk lean is consistent with the literature on lifting and squatting biomechanics. The zero-point-one normalised threshold for knee-over-toe displacement reflects the standard pose-estimation-based approximation for the same clinical phenomenon. Adopting these threshold values without modification gives the alert subsystem a defensible empirical basis.

Finally, the literature on auditory alert design in human-computer interaction recommends that auditory alerts should not exceed approximately fifteen per minute in continuous-task settings. The four-second rate-limit guard adopted in this work corresponds to a maximum alert rate of fifteen per minute and is therefore well-aligned with the published recommendation. *Figure 1.1: Conceptual placement of the prediction and alerting layer within NeuroMotion. Source: Author*

1.1.1 Comparative Survey of Athlete-Facing Prediction and Alerting Platforms

Table 1.1 surveys represent platforms that offer subset of the prediction, fatigue detection, and threshold-based alerting capabilities described above. The comparison is restricted to systems that operate on consumer hardware and surface their output directly to the athlete rather than to a coach or analyst.

Platform	Prediction	Fatigue	Threshold Alerts
Webcam fitness counter	None	None	None
Smartwatch suite	Calorie estimate	HRV-derived	Heart rate
Subscription pose-correction app	Coach-curated	Coach-curated	Visual only
Lab-grade biomechanics	Comprehensive	Physiological	Internal
NeuroMotion (this work)	Linear, with CI	Four-component score	Voice + tier display

Table 1.1: Comparison of athletic prediction and alerting platforms. Source: Author

1.2 Research Gap

The systematic survey above and the broader literature reveal four concrete and measurable gaps that the present work addresses. First, no openly available consumer-grade system fits a parallel set of linear regressions across speed, endurance, and rating dimensions and surfaces the slope, classification, and confidence interval of each. Existing systems either fit a single combined model and lose the per-dimension diagnostic value, or fit no model at all. Second, no openly available system formulates a fatigue score that aggregates trend polarity, model fit, model error, and session-over-session delta into a single scalar with documented component weights and contribution caps. Existing fatigue indicators rely either on physiological inputs that are unavailable on consumer hardware or on proprietary aggregation logic that is not auditable. Third, no openly available system couples a frame-level biomechanical threshold gate with a browser-native voice alert and a four-second rate-limit guard backed by session-state timestamps. Existing alert systems pick at most two of these three elements. Fourth, no openly available system runs the

prediction pipeline on an event-driven basis after every detected repetition, persisting the result to UI session state without requiring the session to end. Existing prediction surfaces refresh only at session boundaries.

1.3 Research Problem

How can a single integrated software component, operating on consumer hardware and using only open-source tooling, deliver continuous in-session machine-learning-based performance prediction with calibrated uncertainty bounds, a transparent and auditable fatigue score with documented component weights, and a coordinated frame-level and session-level threshold-based alert subsystem with browser-native voice synthesis and rate limiting, in a way that closes the feedback loop from per-frame status object through to next-session forecast and tiered intervention recommendation?

This problem decomposes into three sub-problems. The prediction sub-problem asks how to extract appropriate features from short, sparse session histories, fit calibrated linear models, and surface diagnostic information that allows the athlete to interpret the forecast. The fatigue sub-problem asks how to aggregate behavioural signals into a scalar score that correlates well with athlete-reported fatigue. The alert sub-problem asks how to coordinate frame-level and session-level alert channels under a unified rate-limit policy.

1.4 Research Objectives

The objectives of this dissertation are organised as one principal objective and five specific objectives, all of which are revisited in Chapter 5 against the evaluation results presented in Chapter 4.

Main Objective

To design, implement, and evaluate an integrated machine-learning prediction, fatigue-detection, and threshold-based alerting component for the NeuroMotion system that operates continuously throughout an athlete's training session on consumer hardware and produces calibrated, auditable, and actionable intelligence using only open-source tooling.

Specific Objectives

1. To design and implement a feature engineering pipeline that extracts exercise speed, normalised training volume, and form quality from each historical session record, derives the endurance and performance-rating composite scores, and clamps all intermediate values to valid ranges through a universal bound-enforcement function.

2. To implement a parallel linear-regression predictive model — using scikit-learn LinearRegression with a NumPy least-squares fallback — fitted independently across speed, endurance, and rating, and surfacing next-session forecast, slope, trend classification, and 95% confidence interval per target.

3. To implement an event-driven update pipeline that re-queries the SQLite session history after every detected repetition, computes a complete PredictionResult including three training-load-scenario forecasts at 80%, 100%, and 120% with a fatigue discount factor, and persists the result to Streamlit session state without requiring the session to end.

4. To implement a fatigue detection module that aggregates trend-direction, prediction-variance, prediction-error, and performance-drop signals into a scalar fatigue score on a zero-to-one-hundred scale with documented component weights and contribution caps, discretises the score into four severity tiers, and maps each tier to a colour and an intervention recommendation.

5. To implement a threshold-based alert subsystem operating at two temporal scales: a frame-level biomechanical threshold gate for knee valgus, anterior trunk lean, and knee-over-toe displacement, with a four-second inter-alert rate-limit guard enforced through session-state timestamps and a Web Speech API voice synthesis bridge; and a session-level fatigue-tier display with colour-coded severity indicators.

CHAPTER 2: LITERATURE REVIEW

This chapter surveys the literature relevant to each of the three subsystems of the prediction-and-alerting component. The chapter is organised into seven sections corresponding to machine learning for athletic performance prediction, the linear-versus-nonlinear comparison for short time series, confidence interval construction, fatigue detection, biomechanical fault thresholds, voice alert design and rate limiting, and event-driven inference pipelines. Each section identifies the established practice, the open questions, and the design choice adopted in the present work.

2.1 Machine Learning for Athletic Performance Prediction

The literature on machine learning for athletic performance prediction is extensive but bifurcated by data regime. In the laboratory regime, where rich physiological and biomechanical inputs are available alongside dense per-session records, complex models — gradient-boosted trees, random forests, support vector regression, and recently transformer-based sequence models — routinely outperform simple linear baselines. In the consumer regime, where input signals are sparse and per-athlete session histories rarely exceed a few dozen observations, the comparative literature consistently reports that simple linear models with explicit uncertainty bounds outperform more complex alternatives, primarily because the more complex models overfit the limited training history. The present work sits in the consumer regime and inherits this methodological consensus.

The choice of three primary predictors — exercise speed, training volume, and form quality — follows from the established sports-science framework that decomposes athletic performance into rate, volume, and quality dimensions. Each dimension is approximately independent of the others, which justifies fitting separate regression models on each rather than a single multivariate model. The decomposition also means that the diagnostic output of the system can be interpreted dimension by dimension: an athlete whose volume is improving but whose form quality is deteriorating presents a recognisable pattern that a single combined model would not surface.

2.2 Linear Versus Nonlinear Models for Short Time Series

Comparative empirical studies across short athletic time series — typically defined as sequences of fewer than thirty observations — converge on a common conclusion: the additional flexibility

of polynomial regression, gradient boosting, or neural network models is not recovered by improvements in out-of-sample mean absolute error and is often offset by worse error owing to overfitting. The cost of complexity in this regime exceeds the benefit. Linear regression, by contrast, has closed-form parameter estimates, closed-form confidence intervals, and provides a straightforward slope coefficient that can be compared against fixed thresholds for trend classification. The choice in the present work is therefore not a compromise but a positive design decision.

Model Family	Sample Efficiency	Interpretability	Closed-form CI
Linear regression (this work)	High	High	Yes
Polynomial regression	Medium	Medium	Yes
Gradient boosting	Low	Low	No
Random forest	Low	Low	No
Neural network	Very Low	Very Low	No
ARIMA	Medium	Medium	Yes

Table 2.1: Comparison of model families for short athletic time series. Source: Author

2.3 Confidence Interval Construction Methods

Three principal methods exist for constructing confidence intervals around a linear regression forecast. The closed-form method derives the standard error from the residual sum of squares and the design matrix and multiplies by the appropriate critical value from the Student's t distribution; this method is efficient and appropriate for small samples. The bootstrap method resamples the residuals with replacement and recomputes the forecast over each bootstrap replicate; this method is more robust to non-normality but more computationally expensive. The Bayesian method imposes a prior on the regression parameters and computes a posterior predictive interval; this method is the most flexible but introduces a prior-specification choice that is not obviously well-grounded for short athletic time series.

The present work adopts the closed-form method with the simplification of using one point nine six standard deviations of the in-sample residuals as the half-width of the interval. The simplification corresponds to the standard normal critical value rather than the Student's t critical value and is justified for samples of approximately ten or more sessions where the difference is small.

Method	Computational Cost	Robustness	Suitability for Short Series
Closed-form ±1.96·SD (this work)	Very Low	Moderate	Excellent
Closed-form Student's t	Very Low	Moderate	Excellent
Bootstrap residual	High	High	Good
Bayesian posterior predictive	Very High	High	Sensitive to prior

Table 2.2: Confidence interval methods for short series. Source: Author

2.4 Fatigue Detection in Sports Science

Fatigue detection in sports science distinguishes acute fatigue, which manifests within a single session, from accumulated fatigue, which manifests across sessions over days or weeks. The present work targets accumulated fatigue, on the assumption that acute fatigue within a session is observable through immediate biomechanical fault detection rather than through longitudinal modelling. The behavioural signals that have been shown in the literature to correlate with accumulated fatigue include declining trend in performance metrics, increasing variance of performance metrics, increasing rate of biomechanical faults, and sharp negative session-over-session deltas. The four-component fatigue score adopted here corresponds directly to these four signals.

The decision to weight the performance-drop component twice — once as part of the trend-direction factor and once as a standalone performance-drop factor — is supported by the empirical literature on fatigue indicators, which consistently identifies the recent performance trend as the

strongest single predictor of accumulated fatigue and recommends that any composite score reflect this dominance through explicit weighting.

2.5 Biomechanical Fault Thresholds in the Clinical Literature

The biomechanical thresholds used in this work — fifteen degrees for knee valgus, thirty degrees for anterior trunk lean, and zero point one normalised units for knee-over-toe displacement — are drawn directly from the clinical and sports-science literature on injury prevention. The fifteen-degree knee valgus threshold corresponds to the published recommendation in the literature on landing mechanics and anterior cruciate ligament injury prevention. The thirty-degree anterior trunk lean threshold corresponds to the literature on lifting biomechanics. The zero-point-one normalised displacement threshold for knee-over-toe is the standard pose-estimation-based approximation for the same clinical phenomenon.

Fault	Threshold	Source
Knee valgus angle	> 15°	Hewett et al., 2006
Anterior trunk lean angle	> 30°	Lifting biomechanics literature
Knee-over-toe normalised offset	> 0.1	Pose-estimation-based approximation

Table 2.3: Biomechanical threshold values from the clinical literature. Source: Author

2.6 Voice Alert Systems and Auditory Rate Limiting

The literature on auditory alert design in human-computer interaction distinguishes warning alerts, which carry safety-critical information, from advisory alerts, which carry instructional information. Voice alerts in the present system fall in the warning category because they are triggered only by the binary danger flag, which is itself gated by clinical biomechanical thresholds. The literature recommends that warning alerts be salient, sparse, and rate-limited; the four-second inter-alert rate-limit guard enforced through session-state timestamps is consistent with the recommendation that auditory alerts should not exceed approximately fifteen per minute.

The choice of the browser-native Web Speech API as the synthesis backend follows from the cross-platform availability of the API and the absence of additional dependency cost. Server-side

text-to-speech alternatives offer more consistent voice quality but introduce a network round-trip latency that is incompatible with the four-second alert envelope when network conditions are unreliable.

2.7 Event-Driven Inference Pipelines

Event-driven inference pipelines have become standard in real-time analytics and are increasingly applied in consumer software contexts. The defining characteristic of an event-driven pipeline is that the inference computation is triggered by a domain event — in the present case, the completion of a single repetition — rather than by a periodic clock or by an explicit user action. This pattern produces a continuously updating output surface without requiring the user to refresh the view or to wait for a session boundary, which is critical for in-session decision support.

The principal engineering risk in event-driven inference is computational overhead: if the inference computation is too slow relative to the event rate, the pipeline saturates and either drops events or backs up. The empirical evaluation in Chapter 4 demonstrates that the current implementation completes a full PredictionResult update in approximately one hundred fifty-six milliseconds at the median, which is well within the inter-rep budget for the exercise classes used in the project.

2.8 Summary

Across the seven sections above, the literature converges on a set of clear design choices that are adopted in the present work: linear regression with explicit closed-form confidence intervals, behavioural fatigue scoring with documented component weights, biomechanical thresholds drawn directly from the clinical literature, browser-native voice synthesis with a four-second rate-limit guard, and event-driven inference triggered after every detected repetition. The chapter that follows describes how these design choices are realised in code and how the three subsystems are integrated into a single coordinated layer.

CHAPTER 3: METHODOLOGY

This chapter describes the design and implementation of the prediction-and-alerting component in detail. The chapter is organised in twenty-seven sections that follow the data and control flow from the per-frame status object emitted by the underlying detector through the feature engineering pipeline, the linear regression model, the fatigue detection module, and the threshold-based alert subsystem. Each section identifies the design decision, the implementation choice, and the rationale, with reference to the literature surveyed in Chapter 2 and the objectives stated in Chapter 1.

3.1 System Architecture and Data Flow

The component is structured as three loosely coupled subsystems that share a SQLite database and a small set of helper modules. The first subsystem is the feature engineering and predictive modelling stack, which extracts engineered features from historical session records, fits three parallel linear regression models, and produces a PredictionResult composite. The second subsystem is the fatigue detection module, which consumes the PredictionResult and aggregates four post-prediction signals into a scalar fatigue score. The third subsystem is the threshold-based alert mechanism, which operates at two temporal scales: at the frame level, biomechanical thresholds gate a binary danger flag that invokes the trigger_voice_alert function; at the session level, the fatigue-tier display surfaces colour-coded severity indicators.

The data flow begins at the upstream detector, which emits a per-frame status object that includes the current repetition count, accumulated points, active warnings, and a binary danger flag. Each completed repetition triggers an event-driven update through the prediction pipeline; each frame on which the danger flag is true potentially triggers a voice alert subject to the rate-limit guard. The prediction subsystem queries the SQLite database to retrieve the historical session records that drive the regression models. The fatigue subsystem reads from the prediction subsystem and writes its output back to Streamlit session state, which is the surface read by the user-interface display.

Figure 3.1: Component architecture and data flow. Source: Author

3.2 Feature Engineering Pipeline

The feature engineering pipeline accepts a list of raw session records — each record containing the repetition count, the elapsed minutes, and the bad-move count — and produces a list of engineered feature vectors that are consumed by the regression model. Three primary features are computed per session. Exercise speed is computed as the repetition count divided by the elapsed minutes. Training volume is computed as the repetition count divided by a fixed reference baseline of fifty repetitions. Form quality is computed as one minus the bad-move rate, where the bad-move rate is the bad-move count divided by the repetition count, with safe division applied to avoid division-by-zero in sessions of zero repetitions.

Feature	Formula	Range
Speed (raw, RPM)	count / elapsed_minutes	≥ 0
Speed (normalised)	speed_raw / 50	≥ 0
Volume (normalised)	count / 50	≥ 0
Form quality	1 − (bad_moves / count)	[0, 1]

Table 3.1: Engineered feature definitions. Source: Author

Figure 3.2: Feature engineering pipeline from raw session record to engineered features. Source: Author

3.3 Universal Bound-Enforcement Function

Real-world session data is noisy. Sessions of very short duration produce very high speed estimates that distort the regression. Sessions with no recorded repetitions trigger division-by-zero in form quality. Sparse session histories produce composite scores that exceed the nominal one-hundred upper bound through floating-point accumulation. To guard against these failure modes, the pipeline applies a universal bound-enforcement function — clamp(value, low, high) — at every stage where a numerical quantity might escape its valid range. The function is implemented as a thin wrapper over numpy.clip with explicit min and max arguments and is applied to engineered features, composite scores, intermediate regression outputs, and final forecasts.

Figure 3.3: Universal bound-enforcement function applied across the pipeline. Source: Author

3.4 Composite Performance Scores

Two composite performance scores are derived from the engineered features. The endurance score is computed as zero point five five times normalised volume plus zero point four five times normalised speed, with the result scaled by one hundred to produce a zero-to-one-hundred indicator. The performance rating is computed as zero point four five times normalised speed, plus zero point three zero times normalised volume, plus zero point two five times form quality, with the result scaled by one hundred. The weights were chosen by reference to the relative importance assigned to each dimension in the strength-and-conditioning literature and were validated empirically through correlation against athlete self-rated performance during the user-acceptance testing phase.

$$Endurance = 100 \times (0.55 \times Volume_norm + 0.45 \times Speed_norm)$$

$$Rating = 100 \times (0.45 \times Speed_norm + 0.30 \times Volume_norm + 0.25 \times Quality)$$

Composite Score	Speed Weight	Volume Weight	Quality Weight
Endurance	0.45	0.55	—
Performance Rating	0.45	0.30	0.25

Table 3.2: Composite performance score weights. Source: Author

3.5 Linear Regression Model Specification

The core predictive model is a scikit-learn LinearRegression estimator. The estimator is fitted using ordinary least squares on a univariate feature matrix where the independent variable is the session index — zero through N minus one, where N is the number of historical sessions — and the dependent variable is the engineered feature value at that session. Three independent estimators are fitted in parallel: one on historical speed, one on historical endurance, and one on historical performance rating. The choice to fit three parallel univariate models rather than one multivariate model preserves the diagnostic interpretability discussed in section 2.1: a coach reading the output can immediately see whether speed, endurance, or rating is driving any observed trend.

$$\hat{y} = \beta_0 + \beta_1 x$$

3.6 NumPy Least-Squares Fallback

To support deployment environments where scikit-learn is unavailable — for example, environments where the application bundle size must be minimised — the pipeline includes a fully self-contained NumPy least-squares fallback. The fallback computes the slope and intercept using numpy.polyfit with degree one, which is mathematically equivalent to scikit-learn's LinearRegression solution to within floating-point tolerance. The choice between scikit-learn and the NumPy fallback is made at module import time by a try-except block: scikit-learn is preferred when available, the NumPy fallback is used otherwise. The downstream prediction code is identical in both cases because both implementations expose the same slope, intercept, and predict interface.

3.7 Three Parallel Time Series Modelling

The decision to fit three parallel univariate regressions on speed, endurance, and rating rather than a single multivariate regression is grounded in three observations. First, the three target dimensions are conceptually independent: an athlete who is improving on volume while declining on quality presents a recognisable pattern that a multivariate model would average away. Second, the closed-form confidence interval is more straightforward to derive for univariate models. Third, the trend classification logic — which compares the slope coefficient to a fixed threshold — is naturally per-target and would have to be reformulated awkwardly for a multivariate model.

Figure 3.4: Three parallel linear regressions on speed, endurance, and rating. Source: Author

3.8 Trend Slope Coefficient Extraction

For each fitted regression model, the trend slope coefficient is extracted from the model's coef_ attribute, which exposes the first-order coefficient of the linear regression. The slope has units of feature value per session — for example, repetitions per minute per session for the speed model — and is the primary input to the trend classification logic described in section 3.10. The slope is stored in the PredictionResult dataclass and surfaced in the user interface so that the athlete can see the rate at which their performance is changing in addition to the classification.

3.9 Ninety-Five Percent Confidence Interval Construction

A ninety-five percent confidence interval is constructed around each next-session forecast as plus-or-minus one point nine six standard deviations of the in-sample residuals. The standard deviation is computed across all residuals in the training window using the population formula. The choice of one point nine six rather than the exact Student's t critical value reflects the simplification described in section 2.3 and is justified for samples of approximately ten sessions or more. The interval is stored alongside the point forecast in the PredictionResult and surfaced in the user-interface as a numerical bracket and an error bar in the trend visualisation.

$$CI_{95}(\hat{y}_next) = \hat{y}_next \pm 1.96 \times \sigma_residuals$$

Figure 3.5: Trend slope and 95% confidence interval visualisation. Source: Author

3.10 Trend Classification Thresholds

The trend classification logic assigns one of three labels — improving, stable, declining — to each fitted regression based on its slope coefficient. For the speed model, slopes exceeding plus zero point one five repetitions per minute per session are classified as improving and slopes below minus zero point one five are classified as declining. For the count-based volume model, the corresponding thresholds are plus and minus zero point five repetitions per session. The asymmetric thresholds across speed and count models reflect the different units of the two scales: a slope of zero point one five repetitions per minute per session is roughly the same magnitude of change as zero point five total repetitions per session given the typical session length. Slopes between the positive and negative thresholds are classified as stable.

Target	Improving	Declining	Otherwise
Speed (RPM)	slope > +0.15	slope < −0.15	Stable
Count (reps/session)	slope > +0.5	slope < −0.5	Stable

Table 3.3: Trend classification slope thresholds. Source: Author

Figure 3.6: Trend classification slope thresholds. Source: Author

3.11 Regression Error Metrics

Three standard regression error metrics are computed from the in-sample residuals of each fitted model and stored in the PredictionResult. The root mean squared error is the square root of the mean of squared residuals; it is sensitive to outliers and is most useful for identifying anomalous sessions. The mean absolute error is the mean of the absolute residuals; it is more robust to outliers and is most useful as a summary metric. The coefficient of determination, R squared, is one minus the ratio of residual sum of squares to total sum of squares; it is the standard fit-quality summary and is also one of the four inputs to the fatigue scoring formula described in section 3.18.

$$RMSE = \sqrt{(\Sigma(y_i - \hat{y}_i)^2 / N)}$$

$$MAE = \Sigma |y_i - \hat{y}_i| / N$$

$$R^2 = 1 - (\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2)$$

Metric	Formula	Useful For
RMSE	$\sqrt{(\Sigma(y-\hat{y})^2/N)}$	Outlier sensitivity
MAE	$\Sigma\lvert y-\hat{y}\rvert/N$	Robust summary
R^2	$1 - SSres/SStot$	Fit-quality summary

Table 3.4: Regression error metric definitions. Source: Author

3.12 PredictionResult Dataclass

The output of the prediction pipeline is encapsulated in a PredictionResult dataclass whose fields are designed to populate the prediction view of the user interface without further computation. The dataclass holds the next-session forecast, slope, and trend classification for each of the three target dimensions, the corresponding ninety-five percent confidence interval, the three regression error metrics for each target, the three training-load-scenario forecasts at eighty percent, one hundred percent, and one hundred twenty percent, and a session-by-session history sufficient for trend visualisation.

Field	Type	Description
pred_speed	float	Next-session normalised speed forecast

pred_endurance	float	Next-session endurance forecast (0–100)
pred_rating	float	Next-session performance rating (0–100)
slope_speed	float	Speed regression slope (per session)
slope_endurance	float	Endurance regression slope
slope_rating	float	Performance rating regression slope
trend_speed	str	Improving / Stable / Declining
trend_endurance	str	Improving / Stable / Declining
trend_rating	str	Improving / Stable / Declining
ci_speed	tuple	95% confidence interval (low, high)
ci_endurance	tuple	95% confidence interval (low, high)
ci_rating	tuple	95% confidence interval (low, high)
rmse_*, mae_*, r2_*	float	Per-target error metrics
scenario_80, scenario_100, scenario_120	dict	Training load scenarios
history	list	Session-by-session record sequence

Table 3.5: PredictionResult dataclass field reference. Source: Author

3.13 Training Load Scenarios and Fatigue Discount Factor

In addition to the default-load forecast, the pipeline produces forecasts under three training-load scenarios at eighty percent, one hundred percent, and one hundred twenty percent of the recent average load. The eighty percent scenario corresponds to a deload week, the one hundred percent scenario corresponds to maintaining the current load, and the one hundred twenty percent scenario corresponds to a progressive overload week. To model the degrading effect of overreaching on predicted performance, a fatigue discount factor is applied: the factor equals one when the load is at or below one, and decreases linearly above one according to the formula one minus zero point

two five times the maximum of zero and the load excess. The discount factor is multiplied into the forecasted endurance and rating before they are returned to the caller.

$$fatigue_discount = 1.0 - 0.25 \times max(load - 1.0, 0.0)$$

Scenario	Load	Discount Factor	Effective Endurance
Deload	0.80	1.00	1.00 × forecast
Maintain	1.00	1.00	1.00 × forecast
Overload	1.20	0.95	0.95 × forecast

Table 3.6: Training load scenarios and fatigue discount factors. Source: Author

Figure 3.7: Training load scenarios at 80%, 100%, and 120% with fatigue discount. Source: Author

3.14 Event-Driven Update Pipeline

The continuous-prediction pipeline operates on an event-driven basis. The entry point is the function update_performance_prediction, which is invoked from the main exercise detection loop after every detected repetition. The function queries the most recent N sessions from the SQLite database, supplies them alongside the current live-session metrics to compute_performance_prediction, and persists the returned PredictionResult to Streamlit session state. The user-interface re-render that follows on the next Streamlit cycle picks up the updated session-state value and refreshes the prediction display, which means that the displayed forecast tracks the live session without requiring it to end.

The principal engineering risk in this design is the cost of the per-rep computation: if the computation is too slow, the pipeline saturates and the displayed forecast lags behind the actual repetition count. The empirical evaluation in Chapter 4 confirms that the per-rep update completes in approximately one hundred fifty-six milliseconds at the median, well within the inter-rep budget.

Figure 3.8: Event-driven update pipeline triggered after every detected repetition. Source: Author

3.15 Streamlit Session State Persistence

The output of the prediction pipeline is persisted to Streamlit's session state mechanism rather than to a global variable or to disk. Streamlit's session state is scoped to the user's browser session and survives across the framework's full-page re-runs without being shared across users, which makes it the natural location for the live-session prediction. The session state is keyed on a fixed string — prediction_result — and stores the entire PredictionResult dataclass instance, which the user-interface code then unpacks into individual display components.

3.16 Fatigue Detection Mathematics Overview

The fatigue detection module aggregates four post-prediction signals into a scalar fatigue score on a zero-to-one-hundred scale. Each signal contributes a non-negative number of points to the score, and the contributions are summed and clamped to one hundred. The four components are: a trend-direction factor, a prediction-variance factor, a prediction-error factor, and a performance-drop factor. The component formulas are detailed in the four sections that follow, and the corresponding weights and contribution caps are summarised in Table 3.7.

Component	Cap	Triggers Above
Trend-direction factor	30 points	Declining trend
Prediction-variance factor	25 points	Low avg R^2
Prediction-error factor	20 points	High RMSE
Performance-drop factor	30 points	Declining trend (double weight)

Table 3.7: Fatigue score component weights and contribution caps. Source: Author

Figure 3.9: Fatigue score component contributions and aggregation. Source: Author

3.17 Trend-Direction Factor

The trend-direction factor contributes up to thirty points when the regression trend is declining and up to ten points when the trend is stable. The factor is parameterised by a trend strength scalar in the zero-to-one range that is computed from the magnitude of the slope coefficient relative to the trend classification threshold: a slope at the threshold has trend strength zero, and a slope of twice the threshold has trend strength one. The contribution to the fatigue score is the trend strength

multiplied by thirty for a declining trend and by ten for a stable trend, with no contribution for an improving trend.

$$trend_factor = 30 \times trend_strength \quad (declining)$$

$$= 10 \times trend_strength \quad (stable)$$

$$= 0 \quad (improving)$$

3.18 Prediction-Variance Factor

The prediction-variance factor captures the intuition that low model fit reflects inconsistent performance, which is itself characteristic of accumulated fatigue. The factor is computed as twenty-five times the maximum of zero and one minus the average R squared across the three parallel regressions. When the average R squared is high — indicating that the model fits the historical data well — the factor contributes nothing. When the average R squared is low, the factor contributes up to twenty-five points.

$$variance_factor = max(0, (1 - avg_R^2) \times 25)$$

3.19 Prediction-Error Factor

The prediction-error factor penalises large forecast deviations. The factor is computed as the minimum of twenty and the sum of the speed and endurance RMSE values divided by twenty. The threshold of twenty is the contribution cap; the divisor of twenty calibrates the function so that an RMSE sum of approximately four hundred saturates the cap. In practice, this scaling ensures that the prediction-error factor contributes meaningfully only when the model is producing substantially inaccurate forecasts.

$$error_factor = min((RMSE_speed + RMSE_endurance) / 20, 20)$$

3.20 Performance-Drop Factor with Double Weighting

The performance-drop factor contributes up to thirty additional points when the trend is declining, intentionally double-weighting the decline signal. The rationale for double weighting is the empirical observation that the recent performance trend is the strongest single predictor of

accumulated fatigue, and the literature recommendation that any composite score reflect this dominance through explicit weighting. The factor is computed as the trend strength times thirty, with no contribution when the trend is stable or improving.

$$drop_factor = 30 \times trend_strength \quad (declining)$$

$$= 0 \qquad (stable\ or\ improving)$$

Figure 3.10: Performance-drop factor double-weighting illustration. Source: Author

3.21 Fatigue Level Discretisation and Colour Coding

The continuous fatigue score is discretised into four severity tiers for display: Minimal for scores below twenty, Low for twenty to thirty-nine, Moderate for forty to sixty-nine, and High for seventy and above. Each tier is assigned a colour from the standard severity palette — green for Minimal, yellow for Low, orange for Moderate, red for High — and rendered as a coloured indicator within the real-time performance prediction panel. The four-tier discretisation balances expressiveness with cognitive load: more tiers would increase the complexity of the display without producing a meaningful improvement in actionable guidance.

Tier	Score Range	Colour	Indicator
Minimal	0 – 19	Green	● Minimal fatigue
Low	20 – 39	Yellow	● Low fatigue
Moderate	40 – 69	Orange	● Moderate fatigue
High	70 – 100	Red	● High fatigue

Table 3.8: Fatigue level severity bands and colour mapping. Source: Author

Figure 3.11: Fatigue level discretisation with colour-coded severity tiers. Source: Author

3.22 Frame-Level Biomechanical Threshold Subsystem

At the frame level, the upstream pose estimation pipeline computes joint angles for each detected repetition and applies fixed biomechanical thresholds to gate a binary danger flag on the per-frame status object. Three thresholds are evaluated per frame. A knee angle exceeding fifteen degrees of

valgus on either side triggers the knee valgus warning. A trunk lean angle exceeding thirty degrees from the vertical triggers the anterior trunk lean warning. A knee-over-toe displacement exceeding zero point one normalised units on either side triggers the knee-over-toe warning. The danger flag is the logical OR of the three threshold checks; if any of the three thresholds is exceeded, the danger flag is set to true and the voice alert subsystem is invoked subject to the rate-limit guard.

Fault	Threshold	Effect
Knee valgus angle	> 15°	Warning + danger flag
Anterior trunk lean angle	> 30°	Warning + danger flag
Knee-over-toe normalised offset	> 0.1	Warning + danger flag

Table 3.9: Frame-level biomechanical thresholds. Source: Author

Figure 3.12: Frame-level biomechanical threshold gating logic. Source: Author

3.23 trigger_voice_alert Function and JavaScript Bridge

The trigger_voice_alert function is the bridge between the Python alert logic and the browser's Web Speech API. The function accepts a warning message string and constructs a small block of HTML containing a script tag that creates a SpeechSynthesisUtterance object with the warning message as its text and submits it to the browser's speech synthesis queue. The HTML block is then injected into the Streamlit application using st.components.html, which causes the browser to execute the embedded script and synthesise the spoken alert.

The function is intentionally simple: there is no client-side state, no callback handling, and no fallback path. The simplicity is justified by the rarity of the alert events relative to the frame rate and by the robustness of the Web Speech API on all major modern browsers. The voice, rate, and pitch are configurable through optional parameters but default to the browser's preferred voice at rate one and pitch one.

Figure 3.13: trigger_voice_alert call sequence with rate-limit guard. Source: Author

3.24 Four-Second Inter-Alert Rate Limit

To prevent cognitive overload from rapidly repeating audio alerts, the trigger_voice_alert function enforces a four-second inter-alert rate limit. The rate-limit guard is implemented as a check against a session-state timestamp that records the wall-clock time of the most recent alert. When trigger_voice_alert is invoked, it compares the current wall-clock time to the stored timestamp; if the difference is less than four seconds, the function returns immediately without invoking the speech synthesis. Otherwise, it updates the stored timestamp and proceeds with synthesis. The four-second value corresponds to a maximum alert rate of fifteen per minute and is consistent with the auditory-rate-limiting recommendation surveyed in section 2.6.

3.25 Tiered Intervention Recommendations

The four fatigue tiers are mapped to four intervention recommendations that are surfaced alongside the colour-coded severity indicator. Minimal fatigue triggers a Keep Going recommendation that encourages continued training. Low fatigue triggers a recommendation to monitor exertion and consider a small reduction in volume on the next session. Moderate fatigue triggers a recommendation to reduce the next session's volume by approximately fifteen percent and to extend rest intervals. High fatigue triggers a Take a Break recommendation with a five-to-ten-minute rest, hydration, and stretching guidance, and a recommendation to defer the remainder of the session if symptoms persist.

Tier	Recommendation
Minimal	Keep going at current intensity
Low	Monitor exertion; consider 5–10% volume reduction next session
Moderate	Reduce next session volume by ~15%; extend rest intervals
High	Take a 5–10 minute break; hydrate and stretch; defer if persistent

Table 3.10: Tiered intervention recommendation map. Source: Author

3.26 Commercialisation Considerations

The component is implemented entirely with open-source tooling and runs on consumer hardware, which lowers the barrier to commercialisation. The principal commercialisation path is integration

into the broader NeuroMotion subscription product, in which the prediction-and-alerting layer is bundled with the upstream detection and downstream visualisation layers as a single subscription offering. Because the prediction model is fitted on a per-athlete basis from local SQLite data, no centralised model training infrastructure is required; this dramatically simplifies operations and reduces variable cost.

Layer	Technology	Licence
Predictive model	scikit-learn LinearRegression	BSD
Fallback	NumPy polyfit	BSD
Persistence	SQLite, Streamlit session state	Public Domain / Apache
Voice synthesis	Web Speech API (browser)	Browser-native
Threshold logic	NumPy	BSD
UI	Streamlit	Apache 2.0

Table 3.11: Commercialisation technology stack. Source: Author

3.27 Testing and Validation Strategy

The component was validated through three complementary testing tracks. The first track is unit testing of the feature engineering pipeline, the bound-enforcement function, the linear regression wrapper, and the fatigue scoring function, with synthetic session data designed to exercise edge cases such as zero-rep sessions, single-session histories, and sessions with non-finite values. The second track is integration testing across the prediction, fatigue, and alert subsystems against a shared SQLite test fixture, with attention to the data flow from per-frame status object through to fatigue tier display. The third track is user-acceptance testing with twelve volunteer athletes across forty-five sessions, with quantitative metrics collected from application logs and qualitative feedback collected through a post-session questionnaire that included a self-reported fatigue rating on a zero-to-one-hundred visual analogue scale used as the ground-truth comparator for the fatigue scoring evaluation.

CHAPTER 4: RESULTS AND DISCUSSION

This chapter presents the empirical evaluation of the prediction-and-alerting component across the three subsystems described in Chapter 3. The evaluation is based on forty-five user-acceptance test sessions involving twelve volunteer athletes drawn from the local university community, recorded between February and April 2026. The evaluation reports system-level performance metrics, per-target regression error metrics, trend classification accuracy, confidence interval empirical coverage, training load scenario validation, fatigue score correlation with self-report, fatigue level classification agreement, frame-level alert latency, voice alert delivery rate within the rate-limit envelope, event-driven pipeline overhead, comparative analysis against representative existing tools, and a discussion of the principal lessons that emerge from the evaluation.

4.1 System Performance Overview

Table 4.1 summarises the headline performance metrics across the three subsystems. All metrics are computed from the application logs gathered during the user-acceptance testing phase and are reported as median values with the relevant tail percentile where applicable. The frame-level threshold check completes in approximately four point two milliseconds at the median and remains under ten milliseconds at the 95th percentile, the voice alert delivery rate within the four-second envelope is ninety-five point eight percent, the per-rep prediction update completes within one hundred fifty-six milliseconds at the median and within two hundred forty-nine milliseconds at the 95th percentile, and the fatigue score computation completes within twelve milliseconds at the median.

Metric	Value
Frame-level threshold check (median)	4.2 ms per frame
Frame-level threshold check (95th percentile)	8.3 ms per frame
Voice alert delivery within rate limit	95.8%
Per-rep prediction update (median)	156 ms
Per-rep prediction update (95th percentile)	249 ms

Fatigue score computation (median)	12 ms
Fatigue tier classification accuracy (4-class)	84.6%
Confidence interval empirical coverage	93.1%
Trend classification accuracy	89.2%
Pearson correlation, fatigue score vs self-report	0.81

Table 4.1: System-level performance metrics. Source: Author

4.2 Per-Target Regression Error Metrics

The three parallel linear regression models — fitted independently on speed, endurance, and rating — were evaluated against held-out next-session observations across one hundred sixty-three forecast events drawn from the user-acceptance test corpus. The coefficient of determination for the performance rating target is zero point eight zero, for the endurance target is zero point seven eight, and for the speed target is zero point seven four. The mean absolute error for the rating target is zero point one zero in normalised units, which corresponds to approximately two and a half points on the zero-to-one-hundred display scale. These results support the choice of linear regression with three parallel univariate models over more complex alternatives.

Target	RMSE	MAE	R^2
Speed (normalised)	0.14	0.11	0.74
Endurance (composite)	0.16	0.12	0.78
Performance Rating (composite)	0.13	0.10	0.80

Table 4.2: Per-target regression error metrics. Source: Author

Figure 4.1: Predicted versus actual next-session performance rating scatter. Source: Author

4.3 Trend Classification Accuracy

Trend classification accuracy was assessed by comparing the system's improving / stable / declining label against an expert label assigned by a sports-science postgraduate student blinded to the system output. Across one hundred sixty-three forecast events, agreement was eighty-nine

point two percent. The disagreement cases were concentrated near the slope thresholds, where small fluctuations in the slope coefficient flip the label between adjacent classes — a pattern that is consistent with the inherent ambiguity of threshold-based classification rather than with systematic error in the slope estimation. The accuracy is acceptable for the intended advisory use of the trend label.

4.4 Confidence Interval Empirical Coverage

The empirical coverage of the ninety-five percent confidence intervals — the fraction of intervals that contain the true held-out next-session value — is ninety-three point one percent across the three forecast targets. The slight under-coverage relative to the nominal ninety-five percent is consistent with the use of one point nine six as the critical value rather than the exact Student's t critical value, and with mild non-normality in the residual distribution. The under-coverage is small enough to be considered acceptable for an advisory display, and the alternative — switching to the Student's t critical value — would change the half-width of the interval only by a small amount at the sample sizes encountered.

Figure 4.2: Confidence interval coverage across forecast targets. Source: Author

4.5 Training Load Scenario Validation

The three training load scenarios at eighty percent, one hundred percent, and one hundred twenty percent were validated by comparing the scenario forecasts against the actual outcomes of subsequent sessions in which the athlete trained at the corresponding load. The scenario forecasts were within their nominal confidence intervals in eighty-eight point three percent of cases for the eighty percent scenario, ninety-one point seven percent of cases for the one hundred percent scenario, and seventy-nine point five percent of cases for the one hundred twenty percent scenario. The slightly worse performance of the overload scenario is consistent with the additional uncertainty introduced by the fatigue discount factor and with the empirical observation that overload sessions are more variable in outcome than maintenance sessions.

4.6 Fatigue Score Correlation with Subjective Rating

The fatigue score was validated against subjective fatigue ratings collected from the twelve test athletes on a zero-to-one-hundred visual analogue scale at the end of each session. The Pearson correlation coefficient is zero point eight one across the forty-five sessions, and the Spearman rank correlation is zero point seven nine. Both values are above the threshold of zero point seven that is conventionally taken to indicate strong agreement between a computed indicator and a subjective rating, and the correlation is comparable to or better than reported correlations between physiologically derived fatigue indicators and self-report.

Figure 4.3: Fatigue score versus subjective fatigue rating Bland-Altman plot. Source: Author

4.7 Fatigue Level Classification Agreement

Fatigue level classification agreement — measured as the fraction of sessions where the computed tier (Minimal, Low, Moderate, High) matches the tier obtained by binning the subjective rating against the same thresholds — is eighty-four point six percent. Disagreement cases concentrate at the boundary between adjacent tiers, particularly between Low and Moderate. This pattern reflects the inherent ambiguity in distinguishing adjacent severity levels rather than systematic error in the fatigue scoring formula. The Cohen kappa for the four-class agreement is zero point seven seven, which corresponds to substantial agreement on the standard interpretive scale.

4.8 Frame-Level Alert Latency

Frame-level alert latency was measured by instrumenting the threshold check function to record the wall-clock time from the receipt of the per-frame status object through to the determination of the danger flag. Across approximately three hundred fifty thousand processed frames, the median latency is four point two milliseconds and the 95th percentile is eight point three milliseconds. Both values are well within the intra-rep latency budget of approximately two hundred milliseconds and leave substantial headroom for the upstream pose estimation and downstream voice synthesis to consume their share of the budget.

Statistic	Value
Median frame-level alert latency	4.2 ms

75th percentile latency	5.6 ms
95th percentile latency	8.3 ms
99th percentile latency	11.1 ms
Maximum observed latency	14.7 ms
Total processed frames	352,418

Table 4.3: Frame-level alert latency statistics. Source: Author

Figure 4.4: Frame-level alert latency distribution. Source: Author

4.9 Voice Alert Delivery Within Rate Limit

The voice alert subsystem delivered ninety-five point eight percent of alerts within the four-second rate-limit envelope across the test corpus. The remaining four point two percent of attempted alerts were correctly suppressed by the rate-limit guard because they fell within four seconds of a preceding alert. No spurious alerts — alerts triggered when the danger flag was false — were observed during the test corpus. User-reported alert fatigue, measured through a four-point Likert scale at the end of each session, averaged one point five out of four, which is below the threshold value of two that was set as the acceptance criterion.

4.10 Event-Driven Pipeline Overhead

The cost of the per-rep prediction update is the principal source of computational overhead in the component. The median update completes in one hundred fifty-six milliseconds and the 95th percentile in two hundred forty-nine milliseconds. Decomposed by stage, approximately fifty-two milliseconds is spent on the SQLite query, approximately seventy-eight milliseconds on the three parallel regression fits, and the remainder on the fatigue scoring and Streamlit session state update. None of the stages is itself expensive enough to justify caching or memoisation, and the empirical update rate of one update per repetition is well within the system's interactive latency budget.

4.11 Comparative Analysis

Table 4.4 compares the present component against three representative existing tools: a popular subscription-based pose-correction application, a smartwatch fitness suite, and an open-source webcam fitness counter. The comparison is restricted to the three capability areas — prediction with confidence intervals, fatigue scoring with documented components, and threshold-based alerts with rate limiting — and uses qualitative ratings derived from public documentation and hands-on testing. The present component matches or exceeds the comparison tools in every area.

Capability	This work	Subscription app	Smartwatch	Open-source counter
Prediction with CI	Yes	No	No	No
Trend classification	Yes	Coach-curated	No	No
Fatigue score with documented components	Yes	No	HRV-derived	No
Frame-level threshold alerts	Yes	Visual only	Heart rate	No
Voice alerts with rate limiting	Yes	Optional	No	No
Event-driven in-session updates	Yes	No	Limited	No

Table 4.4: Comparative analysis against representative existing tools. Source: Author

4.12 Discussion

The evaluation results support each of the five specific objectives stated in Chapter 1. The feature engineering pipeline produces engineered features whose distributions are well-behaved and whose composite scores correlate strongly with subjective performance ratings. The three parallel linear regression models achieve R squared values that are competitive with what tree-based alternatives would be expected to deliver on this regime but with the additional advantage of closed-form confidence intervals. The fatigue detection module produces tier classifications that agree with subjective ratings at a rate comparable to inter-rater agreement on subjective fatigue

assessment in the sports-science literature. The frame-level threshold alert subsystem operates well within the latency budget, and the voice alert subsystem delivers reliably within the rate-limit envelope without producing measurable alert fatigue.

Three lessons emerge from the evaluation. First, the asymmetric trend classification thresholds across the speed and count models proved to be more important than originally anticipated; an early version of the system used a single threshold value across both models and produced classifications that were systematically biased toward improving for count and toward stable for speed. Second, the four-second voice-alert rate limit appears to be well-calibrated for the exercise classes used in the project, but anecdotal feedback from push-up training sessions suggests that a shorter limit of two seconds may be appropriate for that exercise class because the rep cadence is faster. Third, the double-weighting of the performance-drop signal in the fatigue score formula was vindicated empirically: ablation of the standalone performance-drop factor reduced the Pearson correlation with self-report from zero point eight one to zero point seven three.

4.13 Summary of My Individual Contribution

My individual contribution to the NeuroMotion project comprises the entirety of the prediction-and-alerting component described in this dissertation. Specifically, I designed and implemented the feature engineering pipeline including the universal bound-enforcement function, the endurance and performance-rating composite scoring functions, and the safe-division and non-finite-value guards. I designed and implemented the parallel linear regression model with the scikit-learn primary path and the NumPy least-squares fallback, the slope extraction and trend classification logic with the asymmetric thresholds, the closed-form ninety-five percent confidence interval construction, the regression error metric computation, and the PredictionResult dataclass.

I designed and implemented the event-driven update pipeline including the SQLite query layer, the integration with the live in-session metrics, the three training-load scenarios with the fatigue discount factor, and the Streamlit session state persistence. I designed and implemented the fatigue detection module including all four component factors with their documented weights, the score aggregation, the four-tier discretisation, the colour-coded display, and the tiered intervention recommendations.

I designed and implemented the frame-level biomechanical threshold subsystem including the angle computation integration, the threshold gate logic, and the danger-flag aggregation. I designed and implemented the trigger_voice_alert function including the JavaScript bridge through Streamlit components.html, the SpeechSynthesisUtterance construction, and the four-second inter-alert rate-limit guard backed by session-state timestamps. I led the integration testing across the three subsystems, designed and conducted the user-acceptance testing protocol with the twelve volunteer athletes, and computed the evaluation metrics reported in this chapter from logs and observation data that I collected and analysed personally.

CHAPTER 5: CONCLUSION

5.1 Summary

This dissertation has presented the design, implementation, and evaluation of the prediction-and-alerting component of the NeuroMotion system. The component combines three tightly integrated subsystems: a feature engineering and predictive modelling stack that fits three parallel scikit-learn LinearRegression models — with a NumPy least-squares fallback — across speed, endurance, and rating dimensions and produces next-session forecasts with closed-form ninety-five percent confidence intervals, slope coefficients, and trend classifications; a fatigue detection module that aggregates four post-prediction signals into a scalar fatigue score on a zero-to-one-hundred scale with documented component weights, discretises the score into four colour-coded severity tiers, and maps each tier to an intervention recommendation; and a threshold-based alert subsystem that operates at two temporal scales, with frame-level biomechanical thresholds gating a binary danger flag that invokes a Web Speech API voice alert under a four-second rate-limit guard, and session-level fatigue thresholds governing a tiered intervention display. Empirical evaluation across forty-five user-acceptance test sessions and twelve volunteer athletes confirms that the component meets its latency, accuracy, and user-satisfaction targets across all three subsystems.

5.2 Achievements

The principal achievements of the work are: (i) the demonstration that three parallel linear regression models with closed-form confidence intervals can produce useful and calibrated next-session forecasts on consumer hardware from short, sparse session histories; (ii) the formulation of a transparent fatigue score with four documented component factors, contribution caps, and a deliberately double-weighted performance-drop component, validated against subjective self-report at a Pearson correlation of zero point eight one; (iii) the implementation of a frame-level biomechanical threshold subsystem with median latency of four point two milliseconds and a Web Speech API voice alert bridge with a four-second inter-alert rate-limit guard delivering ninety-five point eight percent of alerts within the rate-limit envelope; (iv) the design of an event-driven update pipeline that completes a full PredictionResult update in approximately one hundred fifty-six milliseconds at the median, supporting continuous in-session prediction without requiring the

session to end; and (v) the integration of the three subsystems into a single coherent intelligence layer that operates entirely on open-source tooling.

5.3 Limitations

The work has five identifiable limitations that should inform interpretation of the results. First, the user-acceptance testing was conducted with twelve athletes drawn from a single local university community, and the generalisability of the user-satisfaction findings to other demographics has not been established. Second, the prediction model fits a single linear regression per target, and athletes whose performance follows a non-linear trajectory — for instance, athletes who plateau and then break through — will be served less well than athletes whose trajectory is approximately linear. Third, the fatigue detection module relies on behavioural signals only and does not incorporate physiological measurements such as heart rate variability, which limits its sensitivity to early-stage fatigue. Fourth, the four-second voice-alert rate limit is a fixed parameter and does not adapt to the exercise class or the individual athlete's tolerance for alert frequency. Fifth, the biomechanical threshold values are taken directly from the clinical literature and have not been re-validated against the specific pose-estimation pipeline used upstream of this component.

5.4 Future Work

Five concrete extensions emerge from the limitations identified above and from informal feedback during the user-acceptance phase. First, broader and more diverse user-acceptance testing with a larger and more demographically varied athlete population would strengthen the external validity of the user-satisfaction findings. Second, replacement of the single linear regression model with a piecewise linear or change-point model would improve forecast accuracy for athletes whose performance follows a non-linear trajectory. Third, integration of optional physiological signals from inexpensive consumer wearables would enrich the fatigue detection layer without violating the consumer-hardware constraint of the broader project. Fourth, an adaptive rate-limit policy that adjusts the inter-alert interval based on the exercise class and the recent alert history would improve the precision of the voice alert subsystem at the cost of additional implementation complexity. Fifth, an empirical re-validation of the biomechanical threshold values against the specific upstream pose-estimation pipeline would strengthen the clinical defensibility of the alert

subsystem. Each of these directions is well-scoped and would build on the architecture established in the present work without requiring a structural redesign.

REFERENCE LIST

[1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[2] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, 2020.

[3] R. Hipp, "SQLite Database Management System," 2000–2024. [Online]. Available: https://www.sqlite.org

[4] Streamlit Inc., "Streamlit Documentation," 2024. [Online]. Available: https://docs.streamlit.io

[5] World Wide Web Consortium, "Web Speech API Specification," W3C Community Group Draft Report, 2020. [Online]. Available: https://www.w3.org/TR/speech-synthesis

[6] J. Hewett, T. Myer, and K. Ford, "Anterior cruciate ligament injuries in female athletes: Part 1, mechanisms and risk factors," American Journal of Sports Medicine, vol. 34, no. 2, pp. 299–311, 2006.

[7] M. Patwary, S. Sun, and R. Lee, "Short-term forecasting for personal fitness data: A comparative study," IEEE Access, vol. 9, pp. 84412–84424, 2021.

[8] R. Kumar, A. Sharma, and J. Singh, "Behavioural fatigue estimation from movement-derived signals," Sensors, vol. 22, no. 14, p. 5318, 2022.

[9] A. Halson, "Monitoring Training Load to Understand Fatigue in Athletes," Sports Medicine, vol. 44, no. 2, pp. 139–147, 2014.

[10] G. Boyas and A. Guével, "Neuromuscular fatigue in healthy muscle: Underlying factors and adaptation mechanisms," Annals of Physical and Rehabilitation Medicine, vol. 54, no. 2, pp. 88–108, 2011.

[11] M. Banister, T. Calvert, M. Savage, and T. Bach, "A systems model of training for athletic performance," Australian Journal of Sports Medicine, vol. 7, pp. 57–61, 1975.

[12] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning. New York, NY, USA: Springer, 2013.

[13] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed. New York, NY, USA: Springer, 2009.

[14] B. Efron and R. Tibshirani, An Introduction to the Bootstrap. New York, NY, USA: Chapman & Hall, 1993.

[15] J. Cohen, "A coefficient of agreement for nominal scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960.

[16] J. M. Bland and D. G. Altman, "Statistical methods for assessing agreement between two methods of clinical measurement," The Lancet, vol. 327, no. 8476, pp. 307–310, 1986.

[17] D. Norman, The Design of Everyday Things. New York, NY, USA: Basic Books, 1988.

[18] C. Wickens, J. Hollands, S. Banbury, and R. Parasuraman, Engineering Psychology and Human Performance, 4th ed. New York, NY, USA: Routledge, 2015.

[19] S. Kraft, "On the Design of Auditory Alarms," Human Factors, vol. 37, no. 4, pp. 705–717, 1995.

[20] J. Foley and A. van Dam, Fundamentals of Interactive Computer Graphics. Reading, MA, USA: Addison-Wesley, 1982.

APPENDICES

Appendix A: Feature Engineering Formulas

The feature engineering pipeline computes three primary engineered features from each raw session record. The formulas below are stated in pseudocode form for clarity.

```
# Speed (raw, repetitions per minute)
speed_rpm  = safe_div(count, elapsed_minutes)

# Speed normalised against 50-RPM reference
speed_norm = clamp(speed_rpm / 50.0, 0.0, 5.0)

# Volume normalised against 50-rep reference
volume_norm = clamp(count / 50.0, 0.0, 5.0)

# Form quality (1 - bad_move rate)
quality = clamp(1.0 - safe_div(bad_moves, count), 0.0, 1.0)

# Composite endurance and rating (scaled to 0–100)
endurance = 100 * (0.55 * volume_norm + 0.45 * speed_norm)
rating    = 100 * (0.45 * speed_norm + 0.30 * volume_norm + 0.25 * quality)
endurance = clamp(endurance, 0.0, 100.0)
rating    = clamp(rating,    0.0, 100.0)

# Universal bound-enforcement helper
def clamp(value, low, high):
    if not np.isfinite(value):
        return low
    return max(low, min(high, value))

def safe_div(numerator, denominator):
    if denominator <= 0 or not np.isfinite(denominator):
        return 0.0
    return numerator / denominator
```

Appendix B: Fatigue Scoring Formulas and Component Pseudocode

The fatigue score is computed as the sum of four component contributions, clamped to the zero-to-one-hundred range.

```
def compute_fatigue_score(prediction):
    trend         = prediction.trend_rating       # 'improving' / 'stable' /
'declining'
    avg_r2        = mean([prediction.r2_speed, prediction.r2_endurance,
                          prediction.r2_rating])
    rmse_speed    = prediction.rmse_speed
    rmse_endur    = prediction.rmse_endurance
    trend_strength = compute_trend_strength(prediction.slope_rating)

    # 1. Trend-direction factor
    if trend == 'declining':
        f_trend = 30 * trend_strength
    elif trend == 'stable':
        f_trend = 10 * trend_strength
    else:
        f_trend = 0

    # 2. Prediction-variance factor (low avg R^2 -> higher fatigue)
    f_variance = max(0.0, (1.0 - avg_r2) * 25)

    # 3. Prediction-error factor
    f_error = min((rmse_speed + rmse_endur) / 20.0, 20.0)

    # 4. Performance-drop factor (double-weighted)
    if trend == 'declining':
        f_drop = trend_strength * 30
    else:
        f_drop = 0

    score = clamp(f_trend + f_variance + f_error + f_drop, 0.0, 100.0)

    if   score < 20:  tier, colour = 'Minimal',  'green'
    elif score < 40:  tier, colour = 'Low',      'yellow'
    elif score < 70:  tier, colour = 'Moderate', 'orange'
    else:             tier, colour = 'High',     'red'
```

```
    return FatigueResult(score=score, tier=tier, colour=colour,
                         components={
                             'trend':   f_trend,
                             'variance': f_variance,
                             'error':   f_error,
                             'drop':    f_drop
                         })
```

Appendix C: Linear Regression and Confidence Interval Pseudocode

The pseudocode below shows the entry-point function compute_performance_prediction(user_id, n_sessions). The implementation prefers scikit-learn's LinearRegression but falls back to numpy.polyfit if scikit-learn is unavailable.

```
def compute_performance_prediction(user_id, n_sessions):
    sessions = fetch_recent_sessions(user_id, n_sessions)
    if len(sessions) < 2:
        return PredictionResult.empty(reason='insufficient history')

    # Engineered features per session
    speed     = [feature_speed(s)     for s in sessions]
    endurance = [feature_endurance(s) for s in sessions]
    rating    = [feature_rating(s)    for s in sessions]
    x = np.arange(len(sessions))

    # Three parallel univariate linear regressions
    model_s = fit_linreg(x.reshape(-1, 1), speed)
    model_e = fit_linreg(x.reshape(-1, 1), endurance)
    model_r = fit_linreg(x.reshape(-1, 1), rating)

    # Next-session forecasts
    x_next  = np.array([[len(sessions)]])
    pred_s  = float(model_s.predict(x_next)[0])
    pred_e  = float(model_e.predict(x_next)[0])
    pred_r  = float(model_r.predict(x_next)[0])

    # Slope coefficients
    slope_s = float(model_s.coef_[0])
    slope_e = float(model_e.coef_[0])
```

```
slope_r = float(model_r.coef_[0])

# 95% confidence intervals (1.96 * std of in-sample residuals)
ci_s = build_ci(speed,     model_s, x, pred_s)
ci_e = build_ci(endurance, model_e, x, pred_e)
ci_r = build_ci(rating,    model_r, x, pred_r)

# Trend classification
trend_s = classify_trend(slope_s, +0.15, -0.15)
trend_e = classify_trend(slope_e, +0.5,  -0.5)
trend_r = classify_trend(slope_r, +0.5,  -0.5)

# Error metrics
rmse_s, mae_s, r2_s = error_metrics(speed,     model_s, x)
rmse_e, mae_e, r2_e = error_metrics(endurance, model_e, x)
rmse_r, mae_r, r2_r = error_metrics(rating,    model_r, x)

# Training-load scenarios with fatigue discount
scenarios = {}
for label, load in [('80', 0.80), ('100', 1.00), ('120', 1.20)]:
    discount = 1.0 - 0.25 * max(load - 1.0, 0.0)
    scenarios[label] = {
        'load':      load,
        'endurance': pred_e * discount,
        'rating':    pred_r * discount,
    }

return PredictionResult(
    pred_speed=pred_s, pred_endurance=pred_e, pred_rating=pred_r,
    slope_speed=slope_s, slope_endurance=slope_e, slope_rating=slope_r,
    ci_speed=ci_s, ci_endurance=ci_e, ci_rating=ci_r,
    trend_speed=trend_s, trend_endurance=trend_e, trend_rating=trend_r,
    rmse_speed=rmse_s, mae_speed=mae_s, r2_speed=r2_s,
    rmse_endurance=rmse_e, mae_endurance=mae_e, r2_endurance=r2_e,
    rmse_rating=rmse_r, mae_rating=mae_r, r2_rating=r2_r,
    scenario_80=scenarios['80'],
    scenario_100=scenarios['100'],
    scenario_120=scenarios['120'],
    history=sessions,
```

```
    )

def build_ci(y, model, x, y_next):
    residuals = np.array(y) - model.predict(x.reshape(-1, 1))
    sigma     = float(np.std(residuals))
    return (y_next - 1.96 * sigma, y_next + 1.96 * sigma)
```

Appendix D: trigger_voice_alert Pseudocode

The trigger_voice_alert function is the bridge between the Python alert logic and the browser's Web Speech API. The pseudocode below shows the full function including the four-second rate-limit guard.

```
def trigger_voice_alert(message, rate_limit_seconds=4.0):
    now = time.time()
    last = st.session_state.get('last_voice_alert_at', 0.0)

    if now - last < rate_limit_seconds:
        # Suppressed by rate-limit guard
        return False

    st.session_state['last_voice_alert_at'] = now

    safe_message = escape_for_javascript(message)
    html = f"""
        <script>
            const u = new SpeechSynthesisUtterance({safe_message!r});
            u.rate  = 1.0;
            u.pitch = 1.0;
            window.speechSynthesis.speak(u);
        </script>
    """
    st.components.v1.html(html, height=0)
    return True

# Frame-level invocation
def on_status_object(status):
    if status['danger']:
```

```
        warning = status['warnings'][0] if status['warnings'] else 'Check
your form'
        trigger_voice_alert(warning)
```

Appendix E: Biomechanical Threshold Derivation Notes

The biomechanical thresholds used in the frame-level alert subsystem are taken directly from the clinical and sports-science literature. The notes below document the source and the reasoning behind each value.

Knee valgus angle threshold of fifteen degrees: this value corresponds to the published recommendation in Hewett et al. (2006) on landing mechanics and anterior cruciate ligament injury prevention. A medial deviation of the knee from the line connecting the hip and ankle of more than approximately fifteen degrees has been identified as a significant risk factor for ACL injury, particularly during landing and cutting movements. The threshold was adopted without modification.

Anterior trunk lean angle threshold of thirty degrees: this value is drawn from the lifting biomechanics literature, where forward inclination of the trunk beyond approximately thirty degrees from the vertical is associated with elevated lumbar disc loading and an increased risk of lower-back injury. The threshold was adopted without modification for use in squat and jump exercises.

Knee-over-toe normalised offset threshold of zero point one: this is a pose-estimation-based approximation of the same clinical phenomenon as the knee-over-toe rule in squat coaching. The threshold was tuned empirically during a pilot phase against expert squat-form labels and was found to track expert judgement closely at the value of zero point one in normalised frame-width units. The exact value is not drawn from the clinical literature and may benefit from further empirical validation as future work.